Machine Learning

Instructor: Hafiz Abdul Rehman

Lecture 04: K Nearest Neighbor

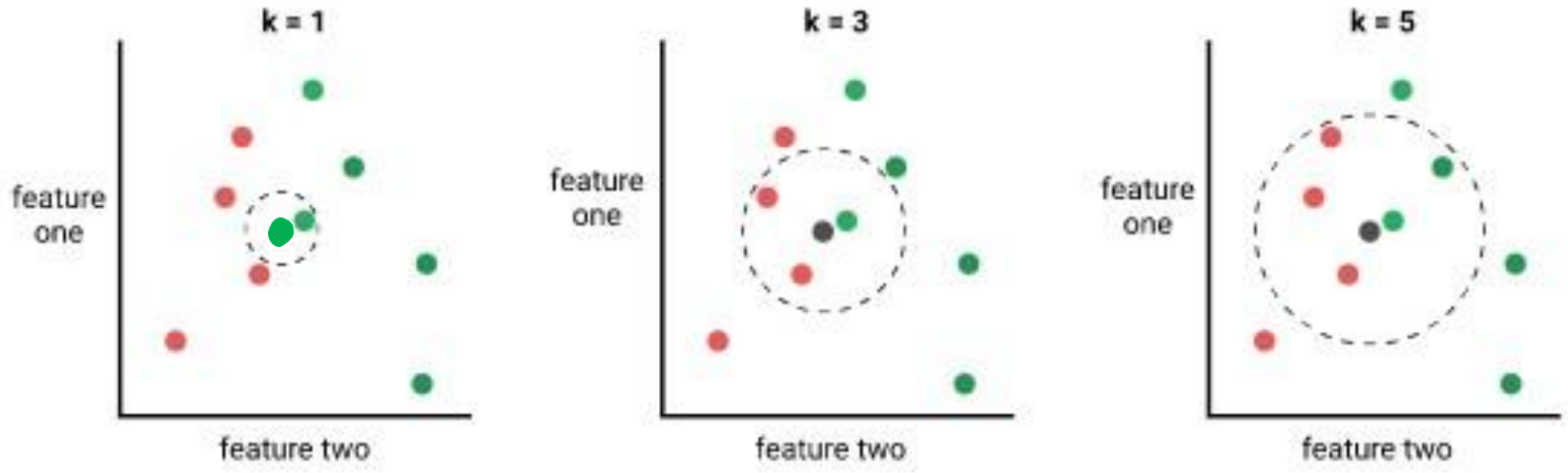
Sources

- **Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell University, Lecture 2, https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote02_kNN.html
- **Nearest Neighbor Methods,** Victor Lavrenko, Assistant Professor at the University of Edinburgh, https://www.youtube.com/playlist?list=PLBv09BD7ez_48heon5Az-TsyoXVYOJtDZ
- **Wiki K-Nearest Neighbors:** https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm
- **Effects of Distance Measure Choice on KNN Classifier Performance -** A Review, V. B. Surya Prasath et al., https://arxiv.org/pdf/1708.04321.pdf
- **A Comparative Analysis of Similarity Measures to find Coherent Documents,** Mausumi Goswami et al. http://www.ijamtes.org/gallery/101.%20nov%20ijmte%20-%20as.pdf
- **A Comparison Study on Similarity and Dissimilarity Measures in Clustering Continuous Data,** Ali Seyed Shirkhorshidi et al., https://journals.plos.org/plosone/article/file?id=10.1371/journal.pone.0144059&type=printable

The K Nearest Neighbors Algorithm

- **Basic idea:** Similar Inputs have similar outputs

- **Classification rule:** For a test input x, assign the most common label amongst its k most similar (nearest) training inputs



Formal Definition

- Assuming x to be our test point, lets denote the set of the k nearest neighbors of x as S_x
- Formally, S_x is defined as

$$\boldsymbol{S_x \subseteq D \; s.t. \; S_x = k}$$
$$\boldsymbol{and}$$
$$\boldsymbol{\forall \; x', y' \in D \setminus S_x,}$$

$$dist(x, x') \geq \max_{(x'', y'' \in S_x)} dist(x, x'')$$

- That is, every point that is in D but not in S_x is at least as far away from x as the furthest point in S_x.
- We define the classifier $h()$ as a function returning the most common label in S_x:

$$\boldsymbol{h(x) = mode(\{y'' : (x'', y'') \in Sx\}),}$$

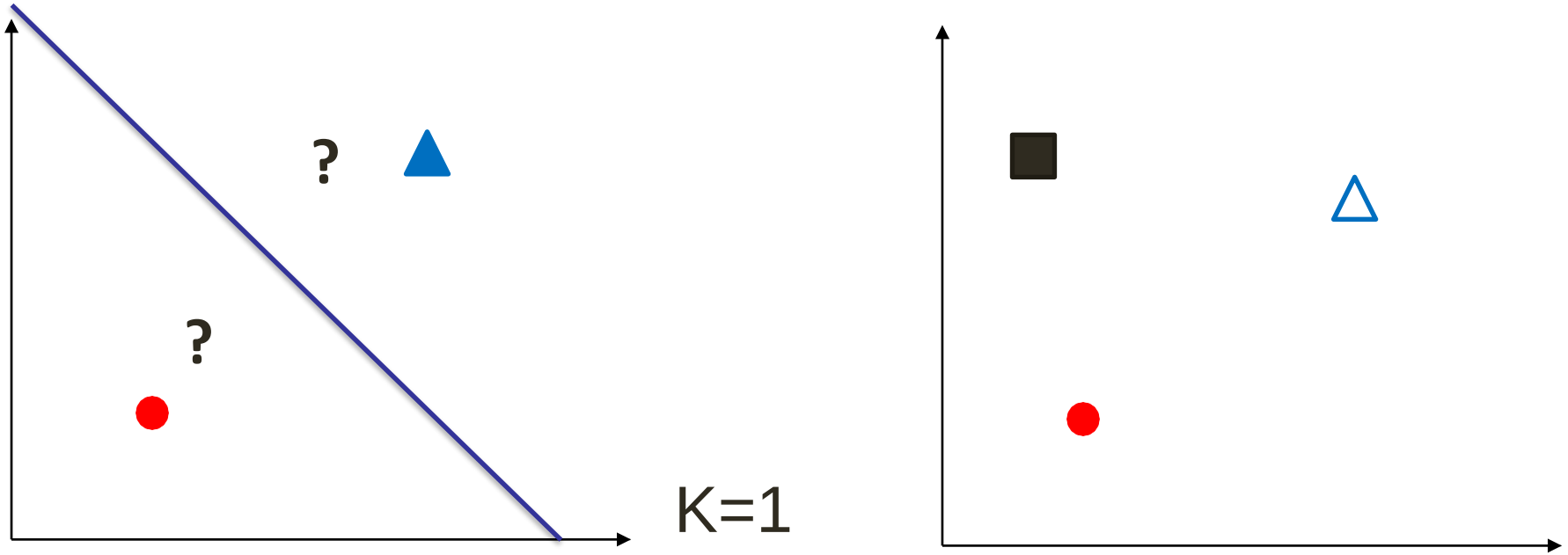
- where mode(·) means to select the label of the highest occurrence.

So, what do we do if there is a draw?

- Keep k odd or return the result of k-NN with a smaller k

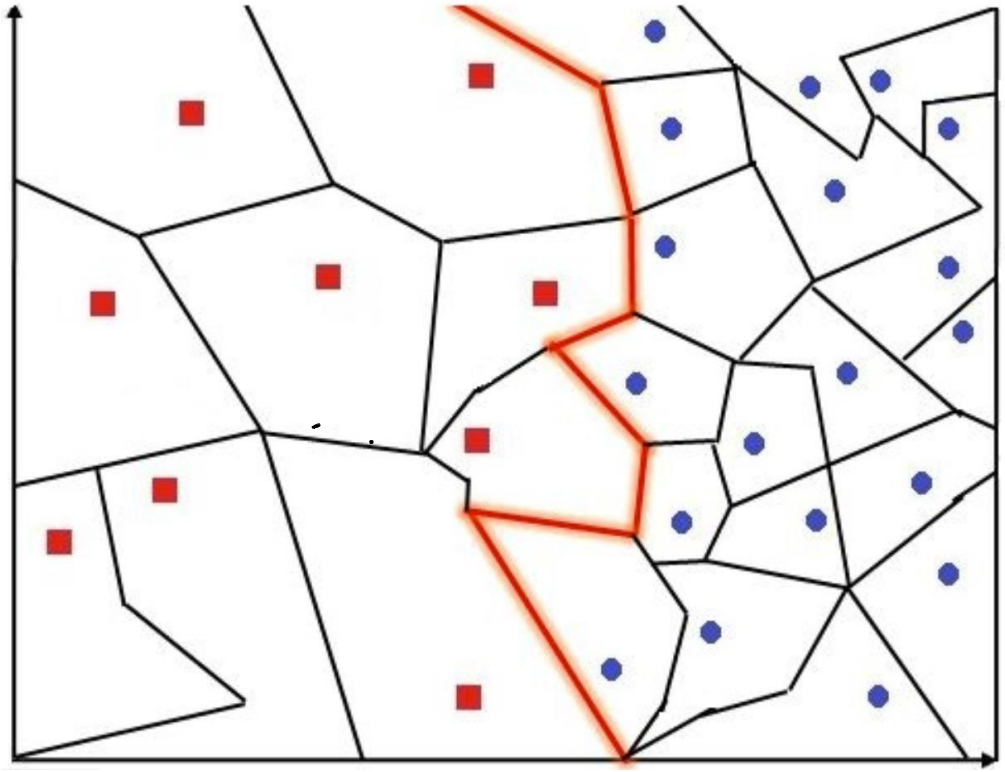
KNN Decision Boundary

- Voronoi Tessellation and KNN decision boundaries



KNN Decision Boundary

- Voronoi Tessellation and KNN decision boundaries



The KNN Algorithm

- A supervised, non-parametric algorithm
  - It does not make any assumptions about the underlying distribution nor tries to estimate it
  - There are no parameters to train like in Logistic/Linear Regression or Bayes
    - **Parameters** allow models to make predictions
  - There is a hyperparameter k, that needs to be tuned
    - **Hyperparameters** help with the learning/prediction process
- Used for classification and regression
  - Classification: Choose the most frequent class label amongst k-nearest neighbors
  - Regression: Take an average over the output values of the k-nearest neighbors and assign to the test point – may be weighted e.g. $\mathrm{w} = {}^{1}$ (d: distance from x)
- An **Instance-based** learning algorithm
  - Instead of performing explicit generalization, form hypotheses by comparing new problem instances with training instances
  - (+) Can easily adapt to unseen data
  - (-) Complexity of prediction is a function of n (size of training data)
- A **lazy** learning algorithm
  - Delay computations on training data until a query is made, as opposed to **eager learning**
  - (+) Good for continuously updated training data like recommender systems
  - (-) Slower to evaluate and need to store the whole training data

Similarity/Distance Measures

- Lots of choices, depends on the problem
- The **Minkowski distance** is a generalized metric form of Euclidean, Manhattan and Chebyshev distances
- The Minkowski distance between two n-dimensional vectors

$$P =< p_1, p_2, \ldots, p_n > \text{ and } Q =< q_1, q_2, \ldots, q_n >,$$

- it is defined as:

$$d_{minkowski}(p,q) = \left(\sum_{i=1}^{n} |p_i - q_i|^a \right)^{1/a}, a \geq 1$$

- a = 1, is the Manhattan distance
- a = 2, is the Euclidean distance
- $a \rightarrow \infty$, is the Chebyshev distance

Constraints on Distance Metrics

- The distance function between vectors p and q is a function
- $d(p, q)$ that defines the distance between both vectors is considered as a metric if it satisfy a certain number of properties:

1. **Non-negativity:** The distance between p and q is always a value greater than or equal to zero

$$\boldsymbol{d(p, q) \geq 0}$$

2. **Identity of indiscernible vectors:** The distance between p and q is equal to zero if and only if p is equal to q

$$\boldsymbol{d(p, q) = 0 \; iff \quad p = q}$$

3. **Symmetry:** The distance between p and q is equal to the distance between q and p.

$$\boldsymbol{d(p, q) = \; d(q, p)}$$

4. **Triangle inequality:** Given a third point r, the distance between p and q is always less than or equal to the sum of the distance between p and r and the distance between r and q

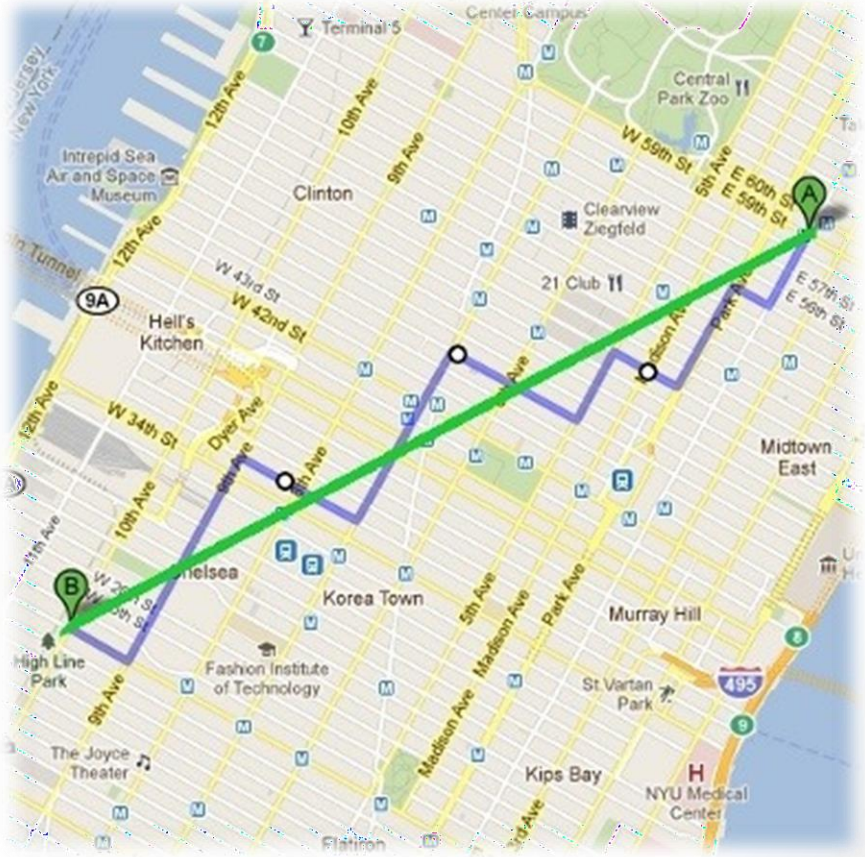
$$\boldsymbol{d(p, q) \leq d(p, r) + d(r, q)}$$

Manhattan Distance

$$\boldsymbol{d_{Man}(p,q) = d(q,p) = |p_1 - q_1| + |p_2 - q_2|, \ldots, |p_n - q_n|}$$

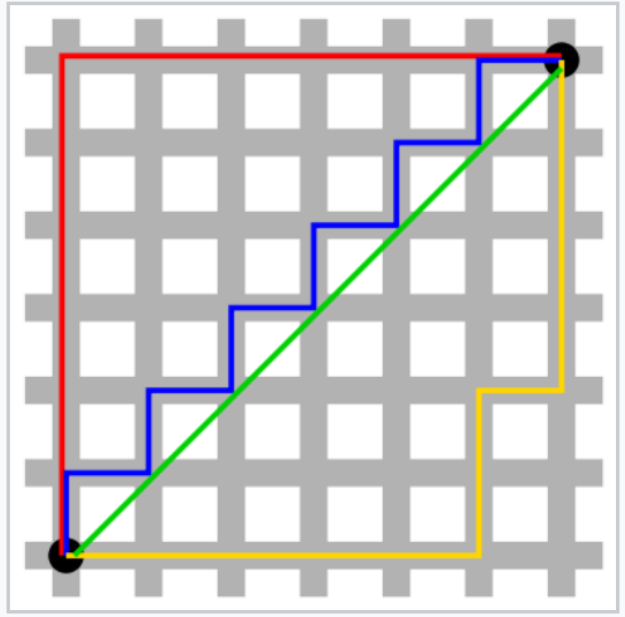
$$d(p,q) = d(q,p) = \sum_{i=1}^{n} | p_i - q_i$$

- The distance between two points is the sum of the absolute differences of their Cartesian coordinates.
  - It is the total sum of the difference between the x-coordinates and y- coordinates.
- Also known as Manhattan length, rectilinear distance, L1 distance or L1 norm, city block distance, snake distance, taxi-cab metric, or city block distance
- Works well for high dimensional data.
  - It does not amplify differences among features of the two vectors and as a result does not ignore the effects of any feature dimensions
  - Higher values of a amplify differences and ignore features with smaller differences



Euclidean vs Manhattan Distance

https://en.wikipedia.org/wiki/Taxicab_geometry



Taxicab geometry versus Euclidean distance: In taxicab geometry, the red, yellow, and blue paths all have the same shortest path length of 12. In Euclidean geometry, the green line has length $6\sqrt{2} \approx 8.49$, and is the unique shortest path.

Euclidean Distance

$$d(p,q) = d(q,p) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2, \ldots, (p_n - q_n)^2}$$

$$d(p,q) = d(q,p) = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2}$$

- Good choice for numeric attributes
- When data is dense or continuous, this is a good proximity measure Downside:

Sensitive to extreme deviations in a single attribute (as it squares

- differences)
  - The variables which have the largest value greatly influence the result.
  - Solution: feature normalization

Chebyshev Distance

Effect of Different Distance Measures in Result of Cluster Analysis, Sujan Dahal

$$d_{Cheb}(p,q) = \lim_{a \to \infty} \left(\sum_{i=1}^{n} |p_i - q_i| \right)^{1/a} = \max_i |p_i - q_i|$$

How?

Assume, $p =< 2,3, \dots, 9 >, q =< 4,6, \dots 10 >$

$$d_{Cheb}(p,q) = \lim_{a \to \infty} (|2-4|^a + |3-6|^a + , \dots + |9-10|^a)^{\frac{1}{a}}$$

$$d_{Cheb}(p,q) = \lim_{a \to \infty} (2^a + 3^a + , \dots + 1^a)^{\frac{1}{a}}$$

Suppose, $a = 2$

$$d(p,q) = (4 + 9 + , \dots + 1)^{\frac{1}{2}}$$

Suppose, $a = 3$

$$d(p,q) = (8 + 27 + , \dots + 1)^{\frac{1}{3}}$$

Suppose, $a = 10$

$$d(p,q) = (1{,}024 + 59{,}049 + , \dots + 1)^{\frac{1}{10}}$$

Now, $a \to \infty$

$$d_{Cheb}(p,q) = \lim_{a \to \infty} \left(\left(\sum_{i=1}^{n} |p_i - q_i| \right) \to \max_i |p_i - q_i| \right)^{\frac{1}{a}}$$

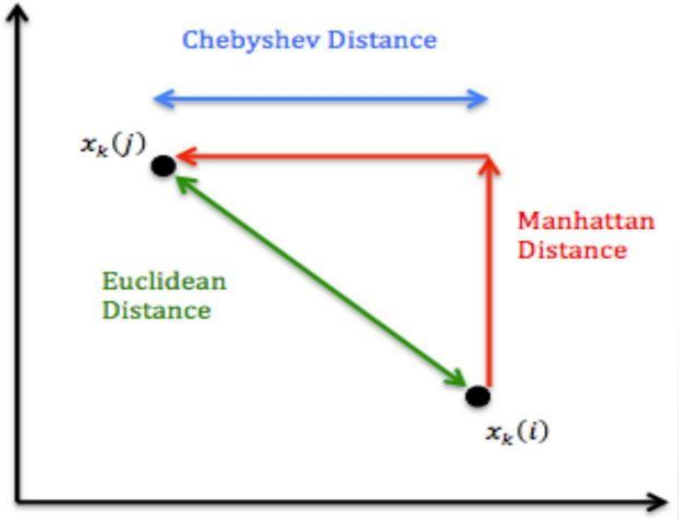
$$d_{Cheb}(p,q) = \lim_{a \to \infty} (\max_i |p_i - q_i|^a)^{\frac{1}{a}}$$

$$\boldsymbol{d_{Cheb}(p,q) = \max_i |p_i - q_i|}$$

Chebyshev Distance

$$d_{Cheb}(p,q) = max|p_i - \ q_i|$$

- For Chebyshev distance, the distance between two vectors is the greatest of their differences along any coordinate dimension
- When two objects are to be defined as “different”, if they are different in any one dimension
- Also called chessboard distance, maximum metric, or $L\infty$ metric



The KNN Algorithm

- **Input:** Training samples D = {(x_1, y_1), (x_2, y_2), … (x_n, y_n)}, Test sample $d = (x, y)$, k. Assume x to be an m-dimensional vector.

- **Output:** Class label of test sample d

1. Compute the distance between d and every sample in D
2. Choose the K samples in D that are nearest to d; denote the set by $S_d \in D$
3. Assign d the label y_i of the majority class in S_d

- **Note:**
  - All action takes place in the test phase, the training phase is essentially to clean, normalize and store the data

KNN Classification and Regression

#	Height (inches)	Weight (kgs)	B.P. Sys	B.P. Dia	Heart disease	Cholesterol Level
1	62	70	110	80	No	150
2	72	90	130	70	No	160
3	74	80	150	70	No	130
4	65	120	140	90	Yes	200
5	67	100	130	85	Yes	190
6	64	110	170	90	No	130
7	69	150	110	100	Yes	250

KNN Classification and Regression

#	Height (inches)	Weight (kgs)	B.P. Sys	B.P. Dia	Heart disease	Cholesterol Level
1	62	70	110	80	No	150
2	72	90	130	70	No	160
3	74	80	150	70	No	130
4	65	120	140	90	Yes	200
5	67	100	130	85	Yes	190
6	64	110	170	90	No	130
7	69	150	110	100	Yes	250
8	**66**	**115**	**145**	**90**	**??**	**??**

KNN Classification and Regression

#	Height (inches)	Weight (kgs)	B.P. Sys	B.P. Dia	Heart disease	Cholesterol Level	Euclidean Distance
1	62	70	110	80	No	150	
2	72	90	130	70	No	160	
3	74	80	150	70	No	130	
4	65	120	140	90	Yes	200	
5	67	100	130	85	Yes	190	
6	64	110	170	90	No	130	
7	69	150	110	100	Yes	250	
8	**66**	**115**	**145**	**90**	**??**	**??**	

KNN Classification and Regression

#	Height (inches)	Weight (kgs)	B.P. Sys	B.P. Dia	Heart disease	Cholesterol Level	Euclidean Distance
1	62	70	110	80	No	150	52.59
2	72	90	130	70	No	160	47.81
3	74	80	150	70	No	130	43.75
4	65	120	140	90	Yes	200	7.14
5	67	100	130	85	Yes	190	16.61
6	64	110	170	90	No	130	15.94
7	69	150	110	100	Yes	250	44.26
8	**66**	**115**	**145**	**90**	**??**	**??**	

KNN Classification and Regression

#	Height (inches)	Weight (kgs)	B.P. Sys	B.P. Dia	Heart disease	Cholesterol Level	Euclidean Distance
1	62	70	110	80	No	150	52.59
2	72	90	130	70	No	160	47.81
3	74	80	150	70	No	130	43.75
4	65	120	140	90	Yes	200	7.14
5	67	100	130	85	Yes	190	16.61
6	64	110	170	90	No	130	15.94
7	69	150	110	100	Yes	250	44.26
8	**66**	**115**	**145**	**90**	**??**	**??**	

KNN Classification and Regression (Fruit)

#	Mass	Width	Height	Color Score	Fruit Name	Fruit Label
1	176	7.4	7.2	0.6	apple	0
2	178	7.1	7.8	0.92	apple	0
3	156	7.4	7.4	0.84	apple	0
4	154	7.1	7.5	0.78	orange	1
5	180	7.6	8.2	0.79	orange	1
6	118	5.9	8	0.72	lemon	2
7	120	6	8.4	0.74	lemon	2
8	**118**	**6.1**	**8.1**	**0.7**	**lemon**	**??**
9	**140**	**7.3**	**7.1**	**0.87**	**apple**	**??**
10	**154**	**7.2**	**7.2**	**0.82**	**orange**	**??**

KNN Classification and Regression (Fruit)

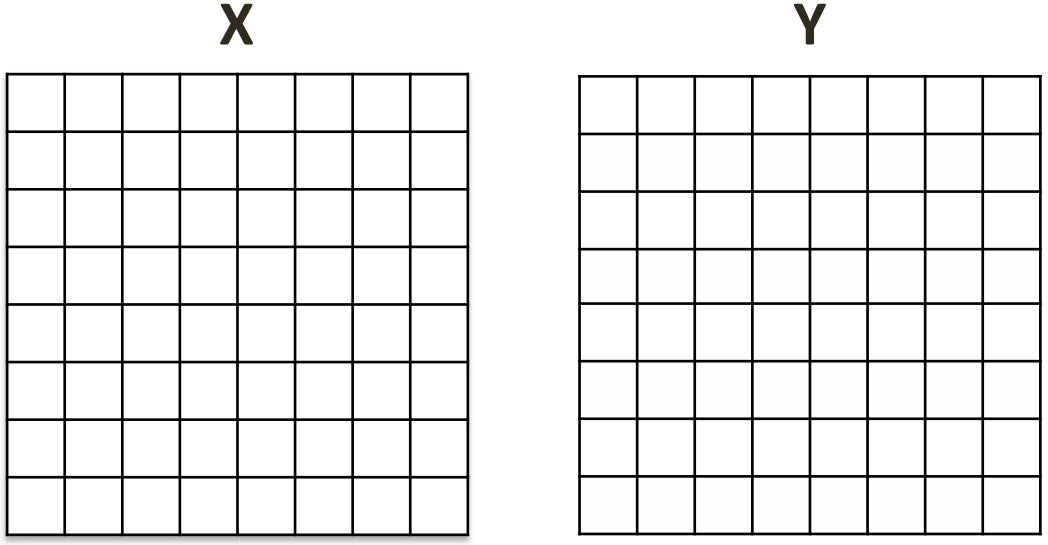
#	Mass	Width	Height	Color Score	Fruit Name	Fruit Label	D8	D9	D10
1	176	7.4	7.2	0.6	apple	0			
2	178	7.1	7.8	0.92	apple	0			
3	156	7.4	7.4	0.84	apple	0			
4	154	7.1	7.5	0.78	orange	1			
5	180	7.6	8.2	0.79	orange	1			
6	118	5.9	8	0.72	lemon	2			
7	120	6	8.4	0.74	lemon	2			
8	**118**	**6.1**	**8.1**	**0.7**	**lemon**	**??**			
9	**140**	**7.3**	**7.1**	**0.87**	**apple**	**??**			
10	**154**	**7.2**	**7.2**	**0.82**	**orange**	**??**			

KNN Classification and Regression (Fruit)

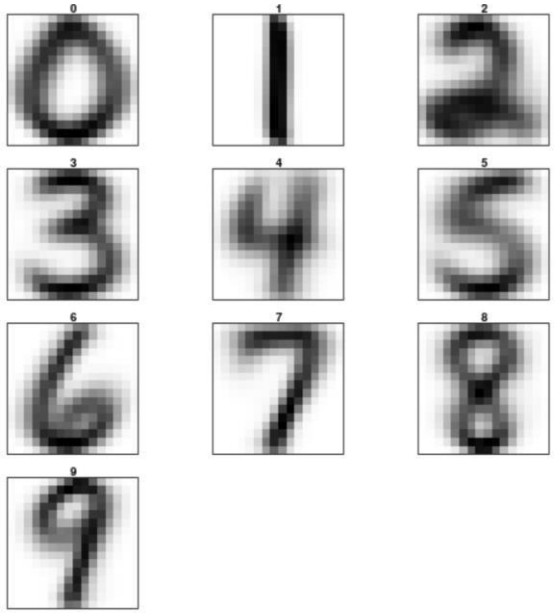
#	Mass	Width	Height	Color Score	Fruit Name	Fruit Label	D8	D9	D10
1	176	7.4	7.2	0.6	apple	0			
2	178	7.1	7.8	0.92	apple	0			
3	156	7.4	7.4	0.84	apple	0			
4	154	7.1	7.5	0.78	orange	1			
5	180	7.6	8.2	0.79	orange	1			
6	118	5.9	8	0.72	lemon	2			
7	120	6	8.4	0.74	lemon	2			
8	**118**	**6.1**	**8.1**	**0.7**	**lemon**	**??**	**2**		
9	**140**	**7.3**	**7.1**	**0.87**	**apple**	**??**		**0**	
10	**154**	**7.2**	**7.2**	**0.82**	**orange**	**??**			**1**

Example: Handwritten digit recognition

- 16x16 bitmaps
- 8-bit grayscale
- Euclidean distances over raw pixels



$$d(p,q) = d(q,p) = \sqrt{\sum_{i=1}^{n}(p_i - q_i)^2}$$



Accuracy

- 7-NN ~ 95.2%
- SVM ~ 95.8%
- Humans ~ 97.5%

http://rstudio-pubs-static.s3.amazonaws.com/6287_c079c40df6864b34808fa7ecb71d0f36.html,
Victor Lavrenko https://www.youtube.com/watch?v=ZD_tfNpKzHY&list=PLBv09BD7ez_48heon5Az-TsyoXVYOJtDZ&index=6

Complexity of KNN

- **Input:** Training samples D = {(x_1, y_1), (x_2, y_2), … (x_n, y_n)}, Test sample $d = (x, y)$, k. Assume x to be an m-dimensional vector.

- **Output:** Class label of test sample d

1. Compute the distance between d and every sample in D
   - **n samples, each is m-dimensional ⇒ $O(mn)$**

2. Choose the K samples in D that are nearest to d; denote the set by $S_d \in D$
   - Either naively do K passes of all samples costing $O(n)$ each time for $O(nk)$
   - Or use the quickselect algorithm (median of medians) to find the kth smallest distance in $O(n)$ and then return all distances no larger than the kth smallest distance. This will accumulate to $O(n)$

- 3. Assign d the label y_i of the majority class in S_d
  - **This is $O\ k$.**

- **Time complexity:** $O(mn + n + k) = O(mn)$, assuming k to be a constant.
- **Space complexity:** $O(mn)$, to store the n, m-dimensional training data samples.

Choosing the value of K – The theory

- **k=1**:
  - High variance
  - Small changes in the dataset will lead to big changes in
  - classification
  - Overfitting
  - Is too specific and not well-generalized
  - It tends to be sensitive to noise
  - The model accomplishes a high accuracy on train set but will be a
  - poor predictor on new, previously unseen data points
- **k= very large (e.g. 100)**:
  - The model is too generalized and not a good predictor on both train and test sets.
  - High bias
  - Underfitting
- **k=n:**
  - The majority class in the dataset wins for every prediction
  - High bias

Tuning the hyperparameter K – the Method

- Divide your training data into training and validation sets.
- Do multiple iterations of m-fold cross-validation, each time with a different value of k, starting from k=1
- Keep iterating until the k with the best classification accuracy (minimal loss) is found

- What happens if we use the training set itself, instead of a validation set? Which k wins?
  - K=1, as there is always a nearest instance with the correct label, the instance itself

KNN – The good, the bad and the ugly

- KNN is a simple algorithm but is highly effective for solving various real life classification problems. Especially when the datasets are large and continuously growing.

- **Challenges:**
  1. How to find the optimum value of K?
  2. How to find the right distance function?

- **Problems:**
  1. High computational time cost for each prediction.
  2. High memory requirement as we need to keep all training samples.
  3. The curse of dimensionality.

Thank You